Craig Ssemakula
COMP4358 - Cloud Computing
Professor Azamhet
April 27, 2026

This report documents Part I of the Comprehensive Project, where I deployed two Linux virtual machines in a single Azure virtual network and configured an Azure Standard Load Balancer to forward traffic from one public IP to each VM through different ports. Apache was installed on both VMs and customized to identify which server responded to the request. Port 8080 maps to VM1 and port 8081 maps to VM2, while the underlying port 80 is not exposed publicly.

1. Architecture Overview

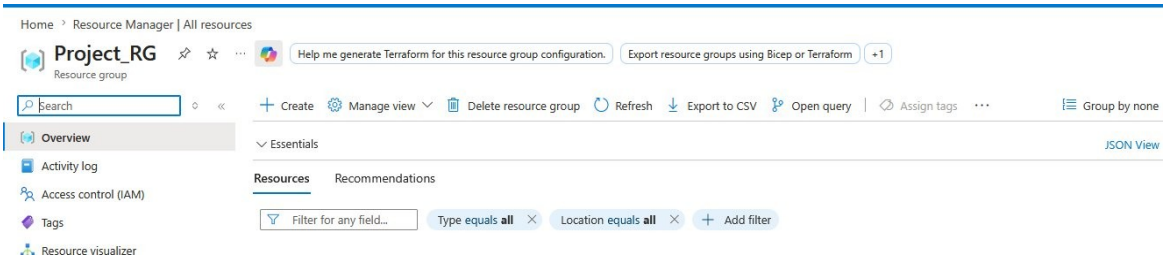
The setup uses a single Standard Public Load Balancer (CJ_LB) sitting in front of two Ubuntu 24.04 VMs (VM1 and VM2) deployed in the same VNet (Cj_VNET) and subnet (CJ_Subnet). Neither VM has its own public IP, so all inbound and outbound traffic passes through Azure-managed components.

Inbound HTTP traffic on the load balancer's public IP is forwarded by two inbound NAT rules: port 8080 to VM1 on port 80, and port 8081 to VM2 on port 80. Outbound traffic from the VMs is handled by a NAT Gateway (CJ_NATGW) attached to the subnet, which is what allowed Apache to be installed and updated in the first place.

2. Configuration Steps

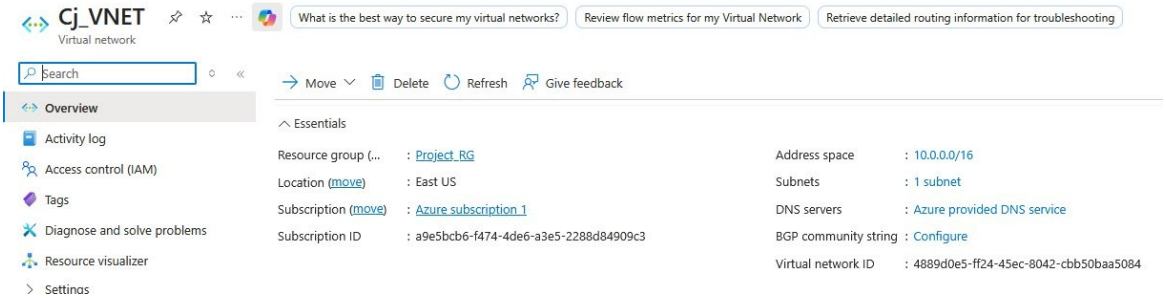
2.1 Resource Group

I created a resource group named Project_RG in the East US region to hold all resources for this project.



2.2 Virtual Network

Inside Project_RG I created a virtual network called Cj_VNET with address space 10.0.0.0/16 and a single subnet (CJ_Subnet) of 10.0.0.0/24.



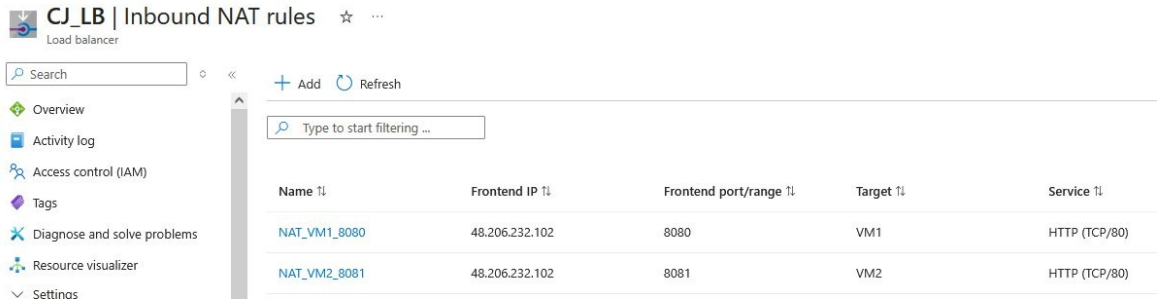
2.3 Virtual Machines

I deployed two Ubuntu 24.04 VMs (VM1 and VM2) into CJ_Subnet. Both were created without public IPs since all access happens through the load balancer. I used the same admin credentials on both so the Apache install steps would be identical.

VM1	···	Azure subscript...	Project_RG	East US	Running	Linux
VM2	···	Azure subscript...	PROJECT_RG	East US	Running	Linux

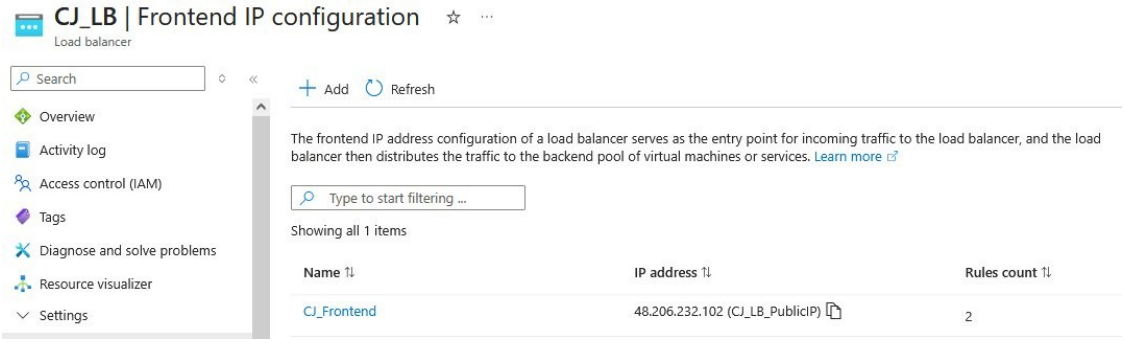
2.4 Load Balancer Inbound NAT Rules

I created a Standard public Load Balancer (CJ_LB) and added two inbound NAT rules to map external ports to each VM. NAT_VM1_8080 forwards the public frontend port 8080 to VM1 on port 80, and NAT_VM2_8081 forwards port 8081 to VM2 on port 80.



2.5 Load Balancer Frontend IP

CJ_LB has a single frontend IP, CJ_LB_PublicIP, which received the address 48.206.232.102. This is the only public entry point into the system, with two rules attached for the two NAT mappings.



2.6 NSG Inbound Rules

Each VM's network security group needed an inbound rule to accept HTTP traffic from the load balancer. I added Allow_LB_HTTP scoped to the AzureLoadBalancer service tag and Allow_HTTP for general TCP port 80.

Priority	Name	Port	Protocol	Source	Destination	Action
100	Allow_LB_HTTP	80	TCP	Internet	Any	Allow
110	Allow_HTTP	80	TCP	Any	Any	Allow

2.7 Apache Installation and Custom Pages

Using the Azure Serial Console for each VM (since they had no public IP), I installed Apache and replaced the default index.html with a custom page identifying the server and its mapped port:

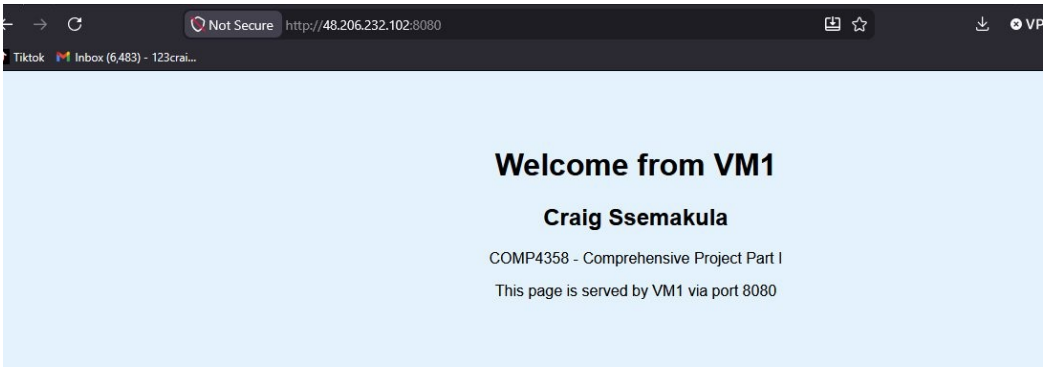
- VM1 page identifies itself as VM1 served via port 8080
- VM2 page identifies itself as VM2 served via port 8081

Both pages include my name and the course code so it is clear which server returned the response.

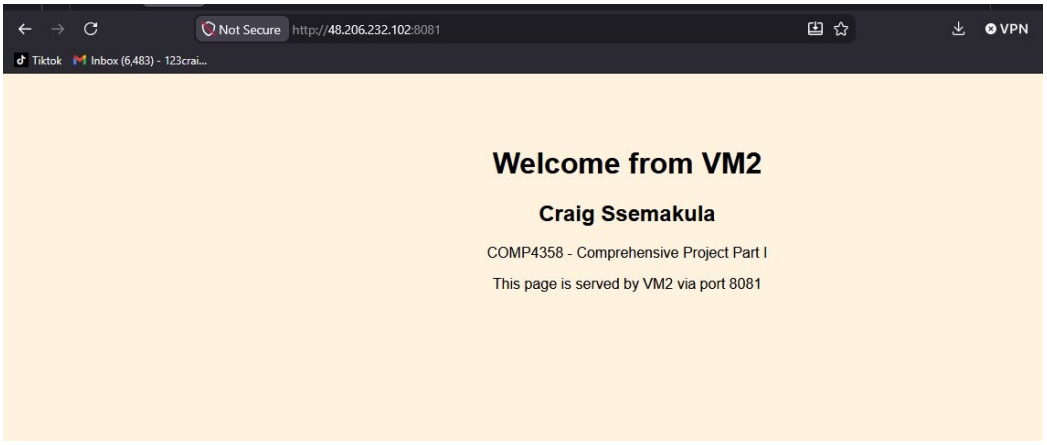
3. Verification

After deployment I tested both NAT rules by browsing to the load balancer's public IP using the two mapped ports. Each port returned the correct VM's custom page.

3.1 VM1 via Port 8080

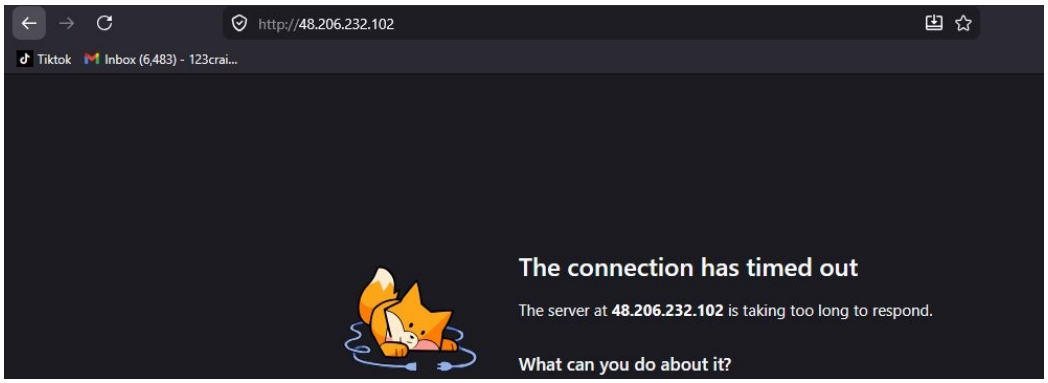


3.2 VM2 via Port 8081



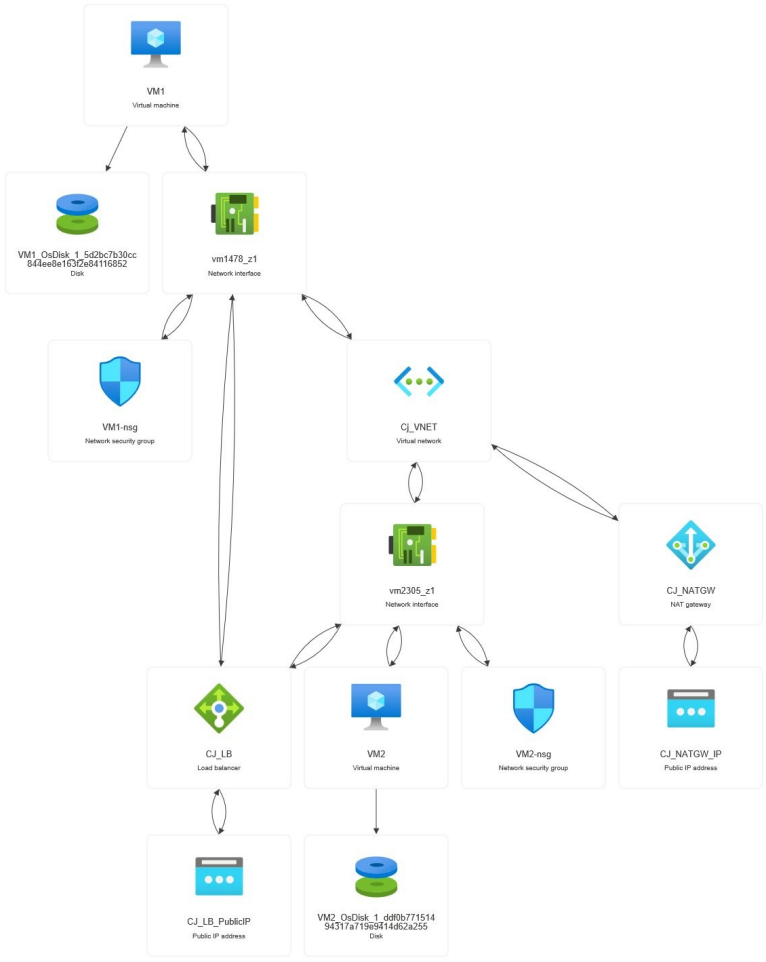
3.3 Port 80 Blocked

To confirm that only the mapped ports are exposed, I also tried browsing to the public IP without specifying a port. The connection timed out, which is the expected behavior since no NAT rule or load balancing rule covers port 80 directly.



4. Resource Topology

The Azure Resource Visualizer shows the full set of resources and how they connect together: the two VMs, their disks and NICs, the shared VNet, the two NSGs, the load balancer with its public IP, and the NAT Gateway providing outbound connectivity to the subnet.



5. Challenges and Solutions

5.1 No Outbound Internet From the VMs

Because both VMs were created without public IPs and the Standard Load Balancer does not provide default outbound connectivity, the first attempt at running apt update inside VM1 timed out trying to reach azure.archive.ubuntu.com. The DNS lookup succeeded but every TCP connection died.

I fixed this by creating a NAT Gateway (CJ_NATGW) and attaching it to CJ_Subnet. Initially the gateway warned that no public IP was attached, so I edited its outbound IP configuration and added CJ_NATGW_IP. Once that was saved, apt update and apt install apache2 -y completed normally on both VMs.

5.2 Index.html Got Overwritten

On the first run I accidentally pasted both the VM1 and VM2 HTML blocks into VM1's serial console, which caused VM1 to serve the VM2 page after Apache was restarted. I rewrote the file with only the VM1 content and restarted Apache, after which VM1 returned the correct page.

5.3 Serial Console Pager Stuck on systemctl Status

Running systemctl status apache2 dropped the console into a pager and any text I typed afterwards (including the next sudo command) was treated as input to the pager. Pressing q exited the pager and returned me to the shell.

6. What I Learned

This project tied together a few pieces I had used separately in earlier labs. The biggest takeaway was the difference between inbound and outbound flows on a Standard Load Balancer. Inbound NAT rules took care of routing the public ports to the right VM, but they do nothing for outbound traffic, which is why the apt install step kept failing until I attached a NAT Gateway.

Mapping different external ports to the same internal port (80) with NAT rules is a clean way to run multiple web servers behind a single public IP without exposing the underlying port directly. It also makes the security boundary easy to reason about, since the only ports a user can hit from the internet are the ones explicitly mapped.

Working through the serial console reinforced how to manage VMs that have no public IP at all. It is slower than SSH but it works regardless of network configuration, which is useful when something on the network side is the actual problem.

7. Conclusion

The Part I requirements were met: two Linux VMs in the same VNet, both running customized Apache servers, accessible only through a single public IP at the load balancer using two distinct ports. Port 8080 reaches VM1, port 8081 reaches VM2, and plain HTTP on port 80 is not exposed.